



Πανεπιστήμιο Θεσσαλίας

Τμήμα Πληροφορικής με Εφαρμογές στην Βιοϊατρική

ΕΞΑΜΗΝΙΑΙΑ ΕΡΓΑΣΙΑ

IMAGE REGISTRATION

ΑΤΟΜΙΚΗ ΓΡΑΠΤΗ ΕΡΓΑΣΙΑ ΣΤΟ

ΜΑΘΗΜΑ ΤΗΣ ΥΠΟΛΟΓΙΣΤΙΚΗΣ ΟΡΑΣΗΣ

ΕΚΠΙΟΝΗΘΗΚΕ ΑΠΟ ΤΗΝ

ΚΩΤΣΙΟΠΟΥΛΟΥ ΜΑΡΙΝΑ, 01824, 5^ο ΕΤΟΣ

ΣΥΝΕΡΓΑΤΙΚΑ ΜΕ

ΣΙΑΚΑΒΑΡΑΣ ΜΑΡΙΟΣ, 01788, 5^ο ΕΤΟΣ

ΤΣΑΚΟΥ ΣΤΑΥΡΟΥΛΑ-ΜΑΡΙΝΑ, 01762, 5^ο ΕΤΟΣ

ΔΙΔΑΣΚΩΝ ΚΑΘΗΓΗΤΗΣ

ΔΕΛΗΜΠΙΑΣΗΣ ΚΩΝΣΤΑΝΤΙΝΟΣ

Η εργασία πραγματοποιήθηκε με τη χρήση του προγράμματος VScode

Λαμία 2023-2024

Πίνακας περιεχομένων

1.	Εισαγωγή	3
2.	Επεξήγηση.....	4
2.1	Συμπλήρωση Πίνακα.....	17
3.	Βιβλιογραφία.....	17

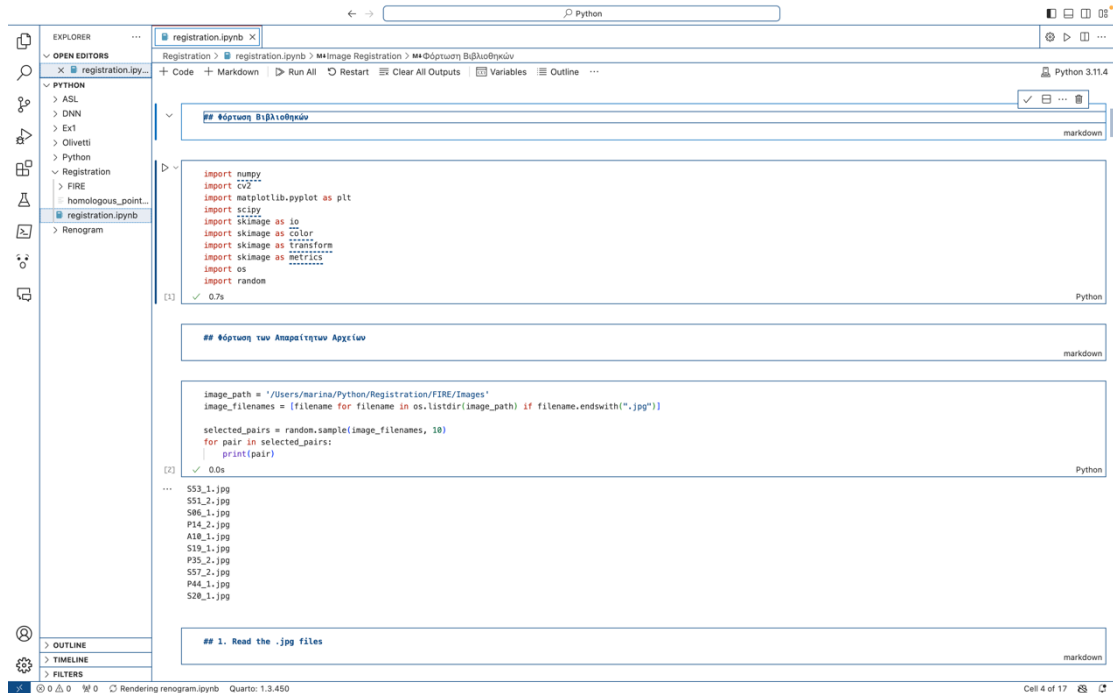
1.Εισαγωγή

Οι γεωμετρικοί μετασχηματισμοί είναι βασική στην επεξεργασία εικόνων, καθώς και στην υπολογιστική όραση. Χρησιμοποιούνται για ευθυγράμμιση εικόνων, διόρθωση παραμορφώσεων, αναγνώριση αντικειμένων και προσαρμογή εικόνων σε συγκεκριμένα πλαίσια. Με τα παραπάνω, μας βοηθούν στη βελτίωση της ακρίβειας και της αξιοπιστίας των εικόνων.

Η εργασία αυτή επικεντρώνεται στη μελέτη γεωμετρικών μετασχηματισμών σε εικόνες αμφιβληστροειδούς με ένα dataset από 134 ζεύγη εικόνων ενός ασθενή. Από αυτό το dataset κρίνεται απαραίτητη η επιλογή 10 ζευγών εικόνων. Σκοπός μας, είναι με τη χρήση των συντελεστών συσχέτισης και της αμοιβαίας πληροφορίας, να ποσοτικοποιήσουμε το ταίριασμα των εικόνων πριν και μετά τον γεωμετρικό μετασχηματισμό.

2. Επεξήγηση

Αρχικά, απαραίτητη κρίνεται η φόρτωση των εικόνων από τον φάκελο Images. Σκοπός μας ήταν η δημιουργία μιας τυχαίας λίστας από 10 ονόματα αρχείων εικόνων που βρίσκονται στον καθορισμένο κατάλογο και η εκτύπωση των ονομάτων των εικόνων που βρίσκονται στην λίστα.



```
# Φόρτωση Βιβλιοθηκών
import numpy
import cv2
import matplotlib.pyplot as plt
import scipy
import skimage as io
import skimage as color
import skimage as transform
import skimage as metrics
import os
import random

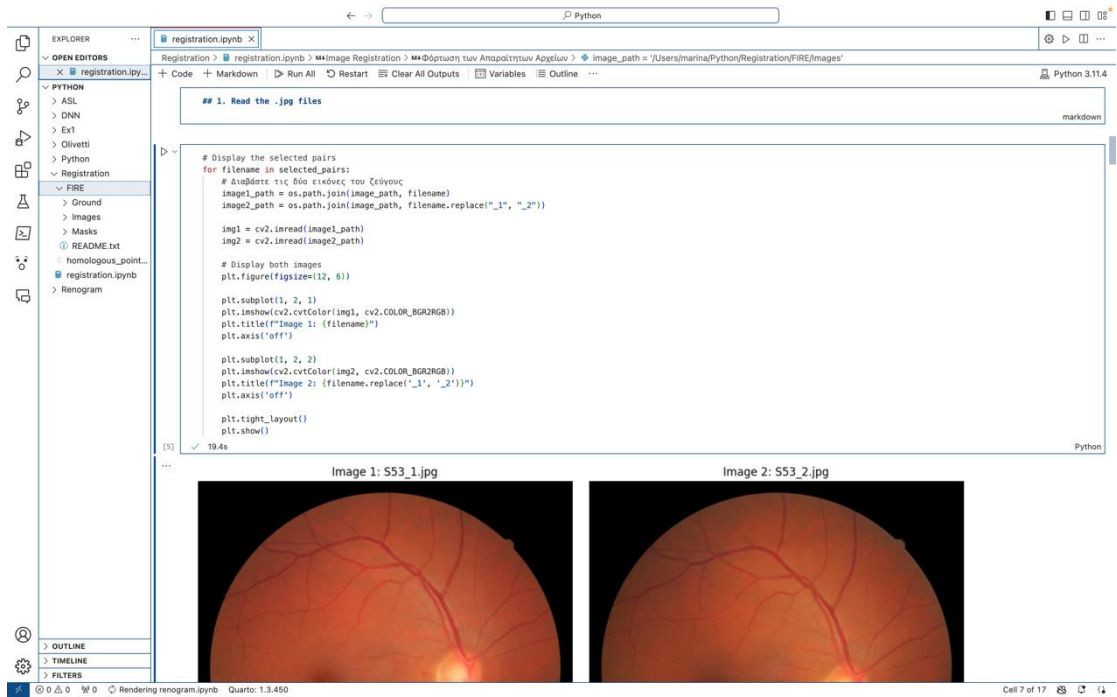
# Φόρτωση των Απαιτούμενων Αρχείων
image_path = '/Users/marina/Python/Registration/FIRE/Images'
image_filenames = [filename for filename in os.listdir(image_path) if filename.endswith(".jpg")]
selected_pairs = random.sample(image_filenames, 10)
for pair in selected_pairs:
    print(pair)
```

553_1.jpg
553_2.jpg
586_1.jpg
P14_2.jpg
A10_1.jpg
S19_1.jpg
P35_2.jpg
S57_2.jpg
P44_1.jpg
S20_1.jpg

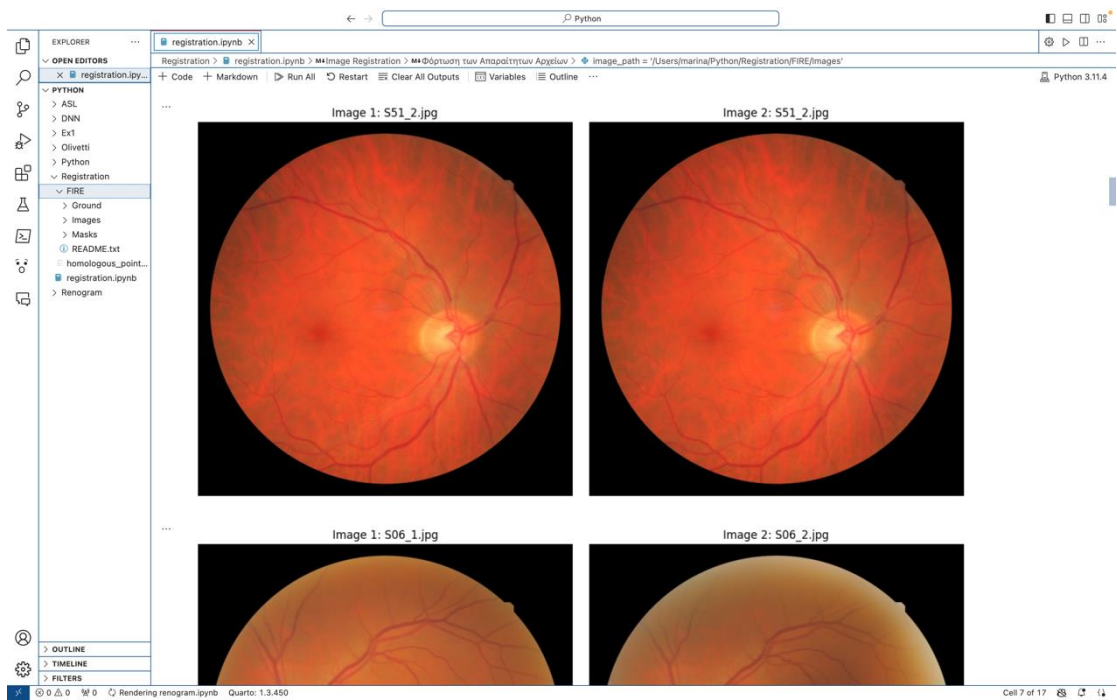
```
# 1. Read the .jpg files
```

Εικόνα 1 Φόρτωση Αρχή και Τυχαία Επιλογή Ζευγών

Στον παρακάτω κώδικα, η μεταβλητή filename, παίρνει τις τιμές των επιλεγμένων ζευγών εικόνων από την λίστα selected_pairs και δημιουργούμε τα ονόματα των αρχείων image1 και image2 βάση του filename. Τέλος, ενώ η πρώτη εικόνα προστίθεται ως έχει, στη δεύτερη εικόνα, χρειάζεται να αντικαταστήσουμε το όνομα της πρώτης με το _2. Αυτό, το κάνουμε καθώς θέλουμε να ελέγξουμε ότι οι δύο εικόνες που ανήκουν σε κάθε ζεύγος, αντιστοιχούν μεταξύ τους βάση των ονομάτων τους.



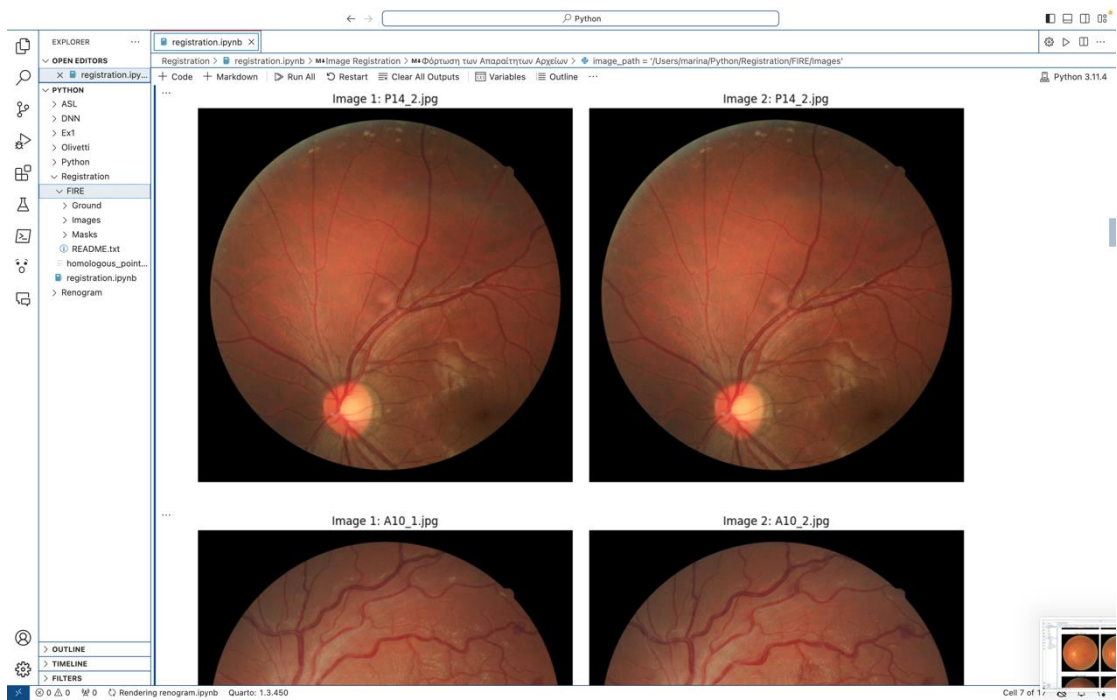
Εικόνα 2 Εμφάνιση των Ζευγών



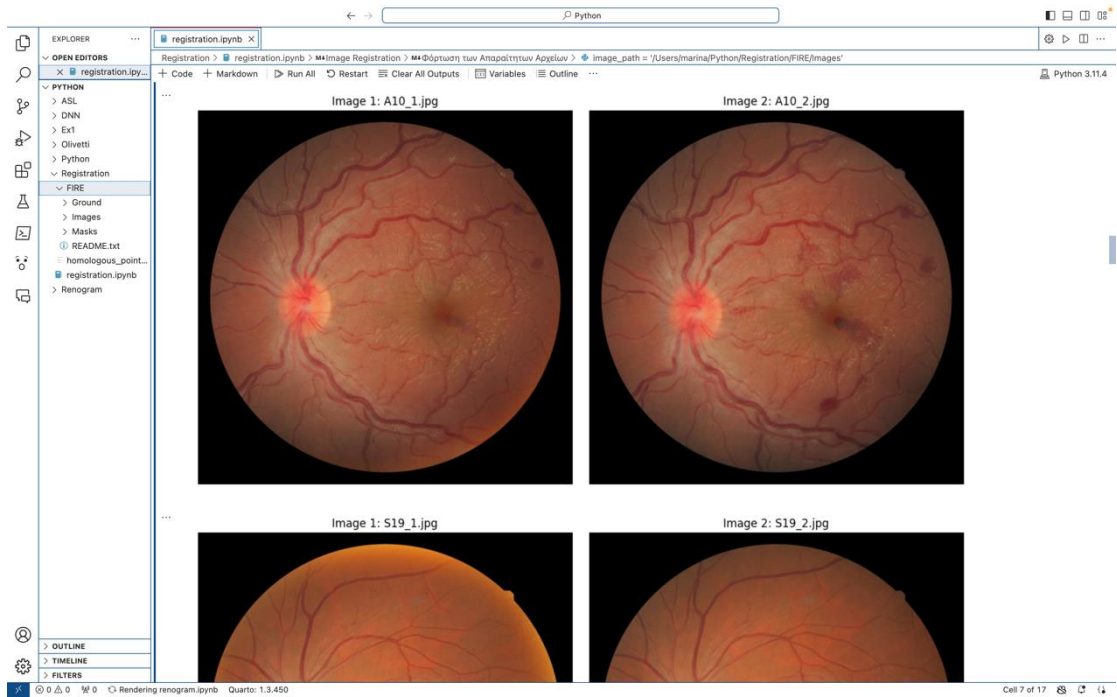
Εικόνα 2.1 Εμφάνιση των Ζευγών



Εικόνα 2.2 Εμφάνιση των Ζευγών



Εικόνα 2.3 Εμφάνιση των Ζευγών



Εικόνα 2.4 Εμφάνιση των Ζευγών



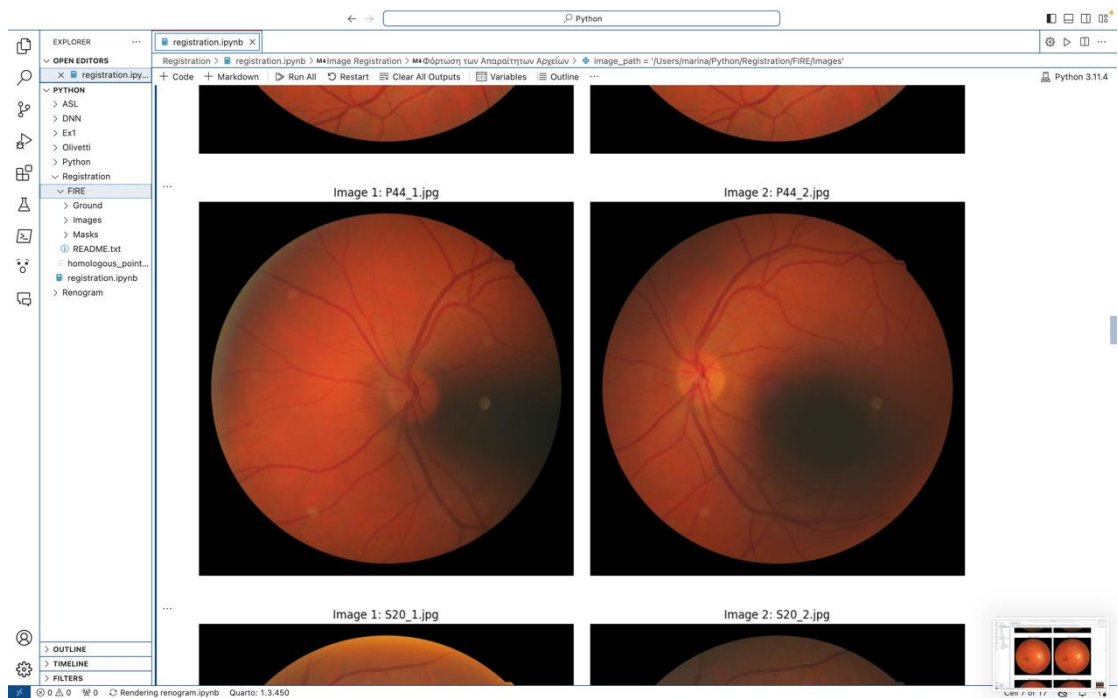
Εικόνα 2.5 Εμφάνιση των Ζευγών



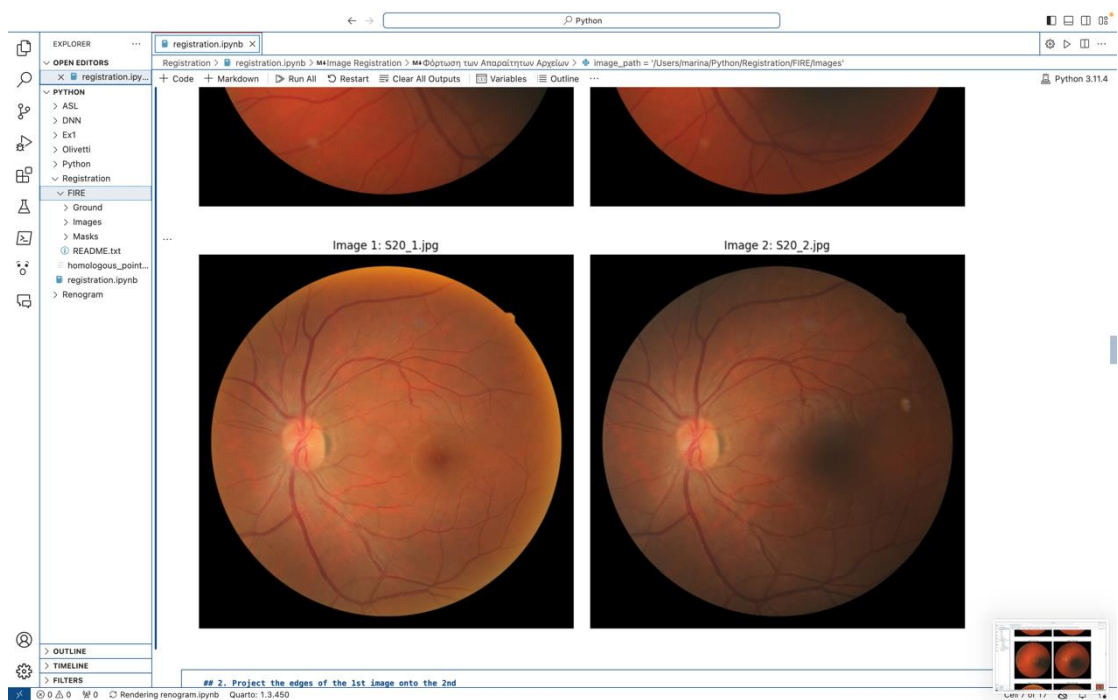
Εικόνα 2.6 Εμφάνιση των Ζευγών



Εικόνα 2.7 Εμφάνιση των Ζευγών

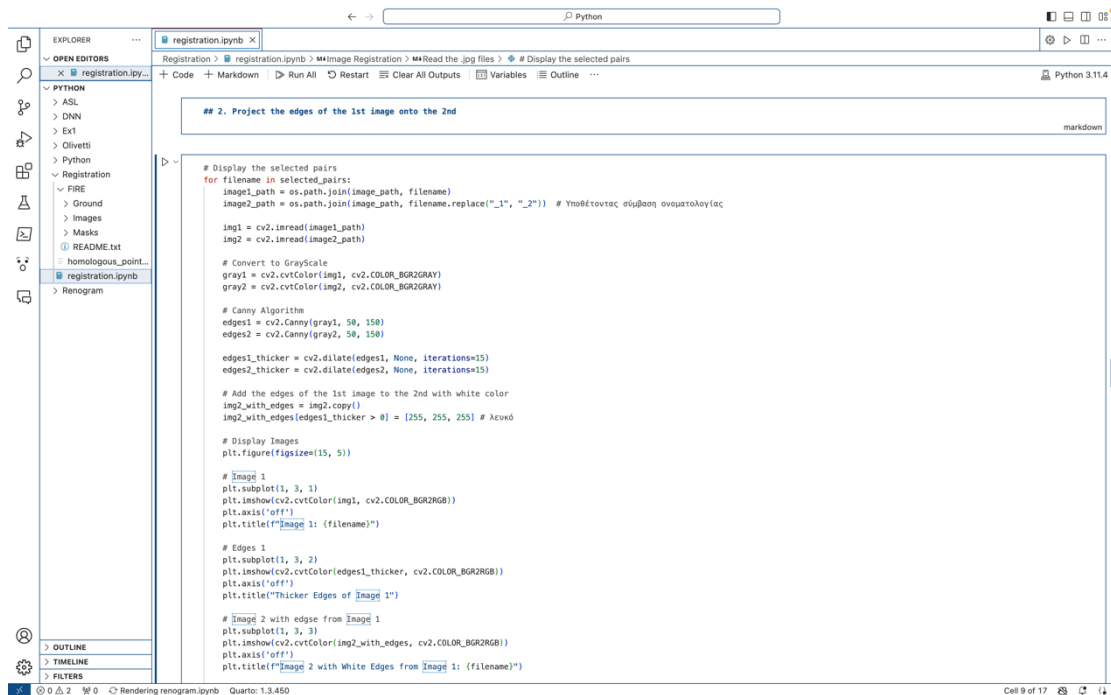


Εικόνα 3Εμφάνιση των Ζευγών



Εικόνα 2.9 Εμφάνιση των Ζευγών

Στον παρακάτω κώδικα, διαβάζουμε τις εικόνες που ανήκουν στο κάθε ζεύγος και τις μετατρέπουμε σε κλίμακα του γκρι καθώς, θέλουμε να εστιάσουμε στην φωτεινότητα των περιοχών, χωρίς να επηρεάζεται το αποτέλεσμα από το χρώμα. Έπειτα, χρησιμοποιούμε τον αλγόριθμο Canny για την ανίχνευση των ακμών. Ουσιαστικά με τη χρήση του αλγορίθμου αυτού, εφαρμόζουμε φίλτρα θορύβου για τη μείωση των τυχαίων διακυμάνσεων, υπολογίζουμε την κλίση για να μπορέσουμε να βρούμε τις περιοχές με μεγάλη αλλαγή στην φωτεινότητα, καταφλιώνουμε για την εύρεση των τιμών που θα αποτελούν τις ακμές. Στην προκειμένη περίπτωση, χρησιμοποιούμε ως κατώτατο κατώφλι το 50, δηλαδή όσες περιοχές βρίσκονται κάτω από το όριο αυτό θα αγνοούνται. Ενώ ανώτατο κατώφλι έχουμε ορίσει το 150, δηλαδή όσες περιοχές έχουν ένταση μεγαλύτερη αυτού του ορίου θεωρούνται ακμές και θα εμφανίζονται στο τελικό αποτέλεσμα. Αυτά είναι τα βασικά βήματα του αλγορίθμου Canny που μας χρειάζονται για να βρούμε έπειτα τον γεωμετρικό μετασχηματισμό.



```
# 2. Project the edges of the 1st image onto the 2nd

# Display the selected pairs
for filename in selected_pairs:
    image1_path = os.path.join(image_path, filename)
    image2_path = os.path.join(image_path, filename.replace("_1", "_2")) # Υποθέτουμε σύμβαση ονοματολογίας

    img1 = cv2.imread(image1_path)
    img2 = cv2.imread(image2_path)

    # Convert to Grayscale
    gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

    # Canny Algorithm
    edges1 = cv2.Canny(gray1, 50, 150)
    edges2 = cv2.Canny(gray2, 50, 150)

    edges1_thicker = cv2.dilate(edges1, None, iterations=15)
    edges2_thicker = cv2.dilate(edges2, None, iterations=15)

    # Add the edges of the 1st image to the 2nd with white color
    img2_with_edges = img2.copy()
    img2_with_edges[edges1_thicker > 0] = [255, 255, 255] # λευκό

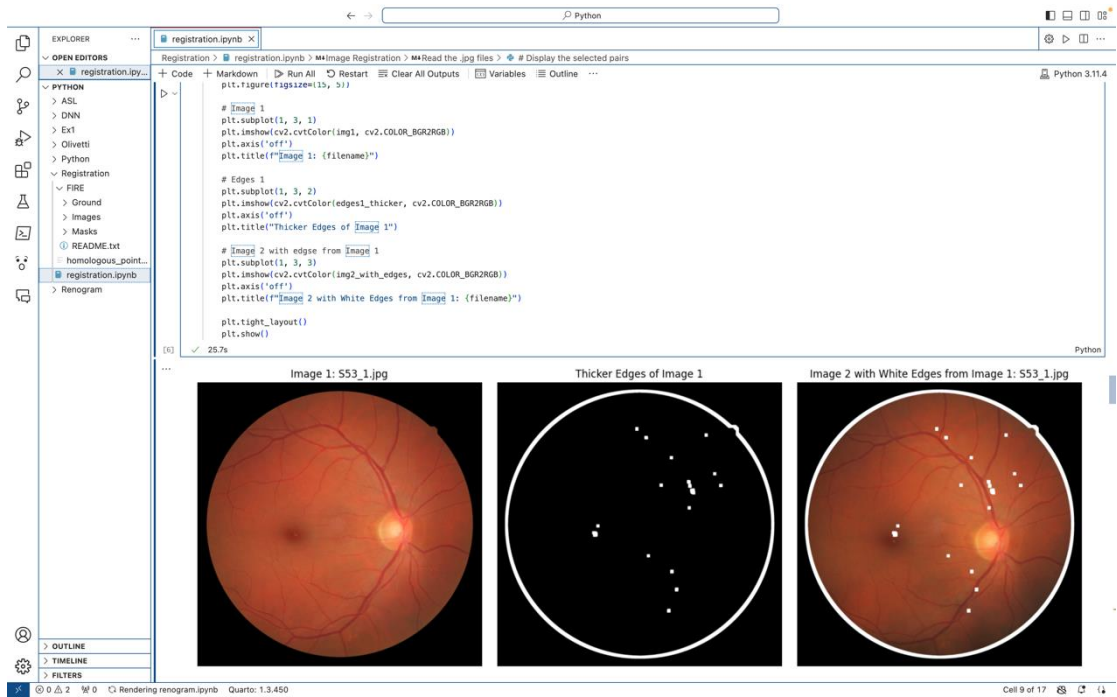
    # Display Images
    plt.figure(figsize=(15, 5))

    # Image 1
    plt.subplot(1, 3, 1)
    plt.imshow(cv2.cvtColor(img1, cv2.COLOR_BGR2RGB))
    plt.axis('off')
    plt.title(f"Image 1: {filename}")

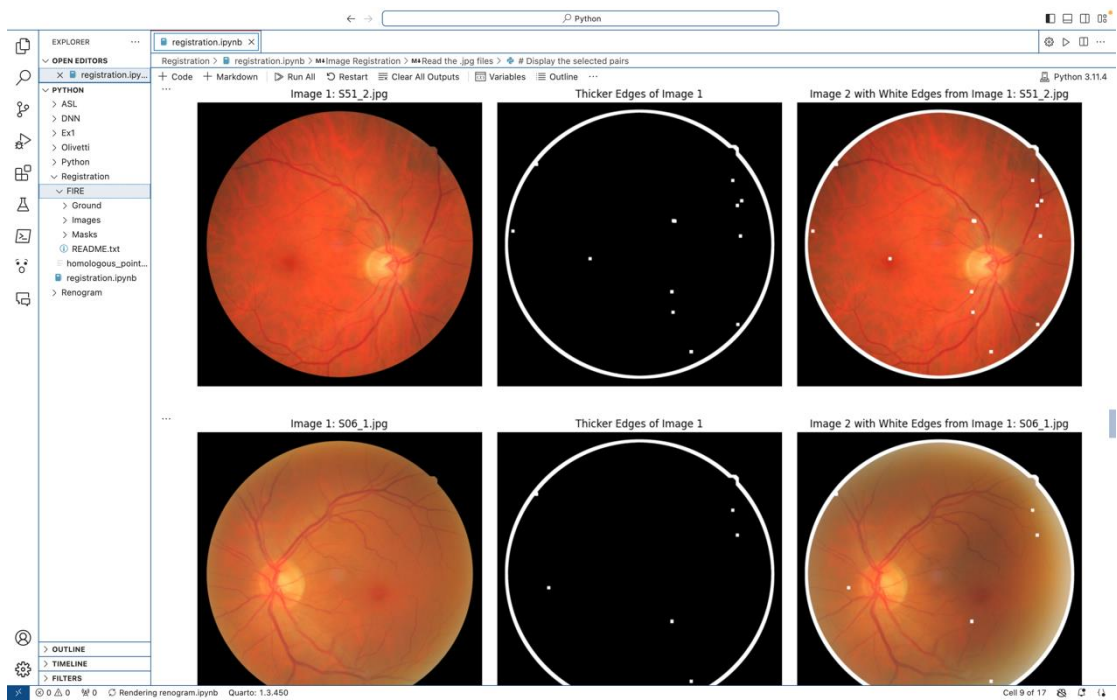
    # Edges 1
    plt.subplot(1, 3, 2)
    plt.imshow(cv2.cvtColor(edges1_thicker, cv2.COLOR_BGR2RGB))
    plt.axis('off')
    plt.title(f"Thicker Edges of Image 1")

    # Image 2 with edge from Image 1
    plt.subplot(1, 3, 3)
    plt.imshow(cv2.cvtColor(img2_with_edges, cv2.COLOR_BGR2RGB))
    plt.axis('off')
    plt.title(f"Image 2 with White Edges from Image 1: {filename}")
```

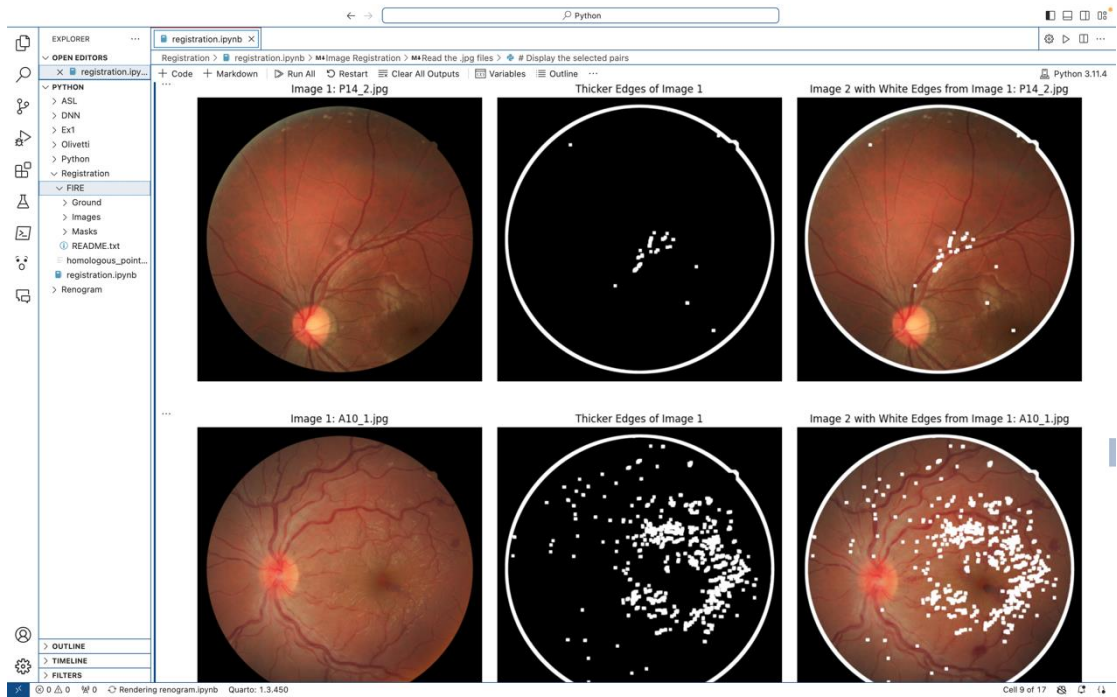
Εικόνα 3 Κώδικας Ακμών



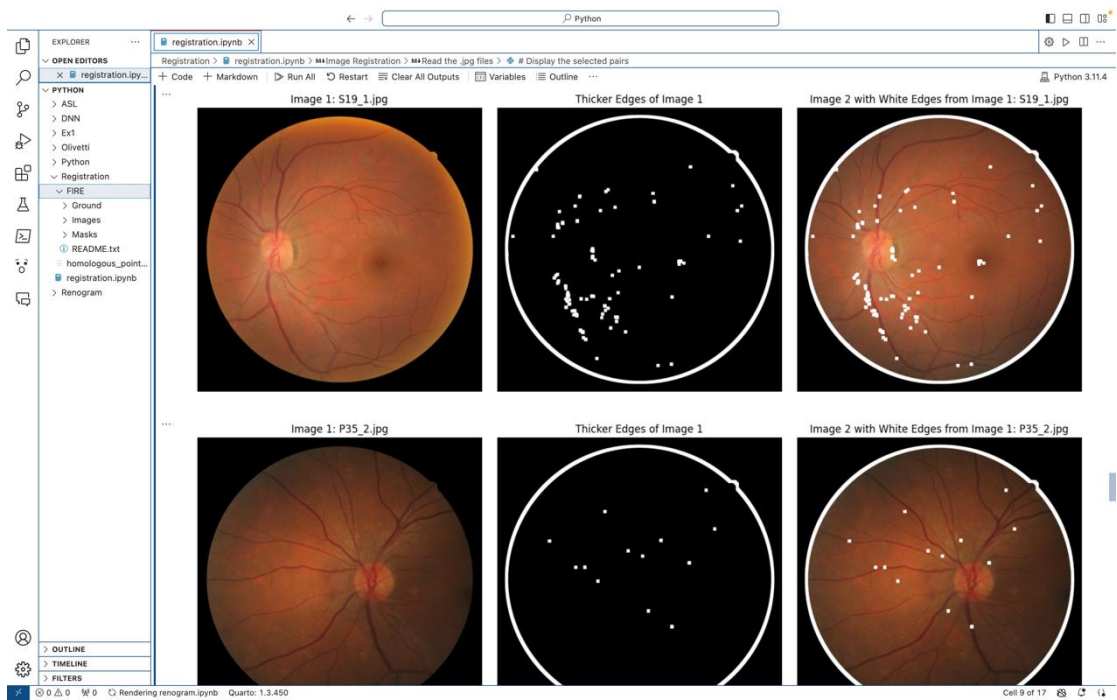
Εικόνα 3.1 Κώδικας Ακμών και Εμφάνιση τους



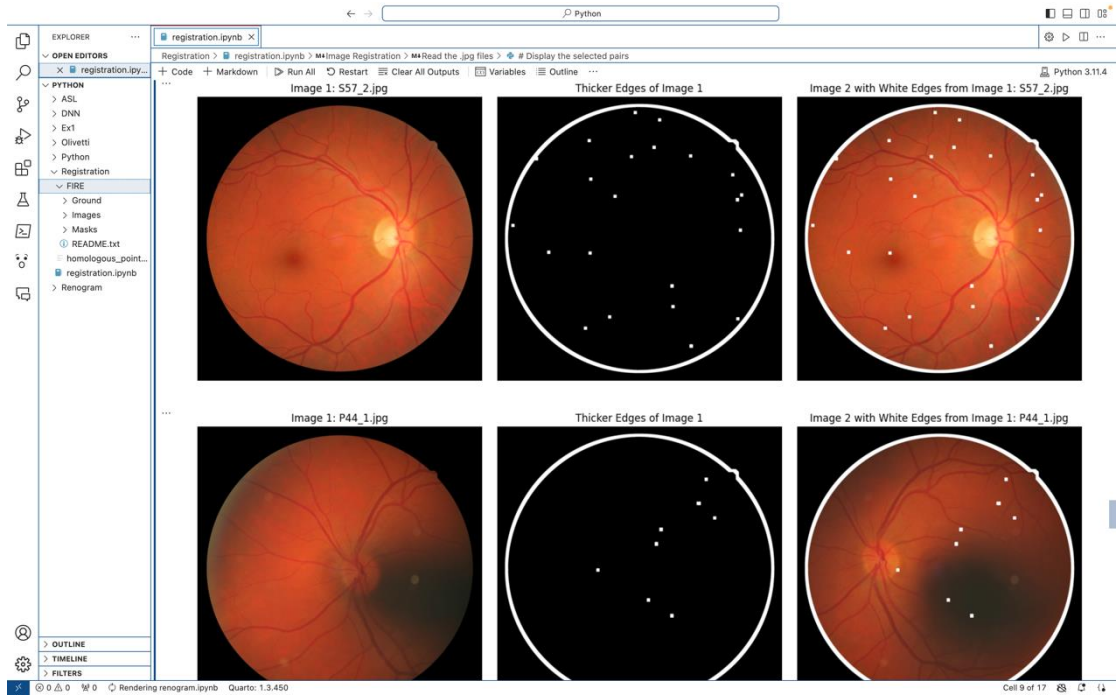
Εικόνα 3.2 Εμφάνιση Ακμών



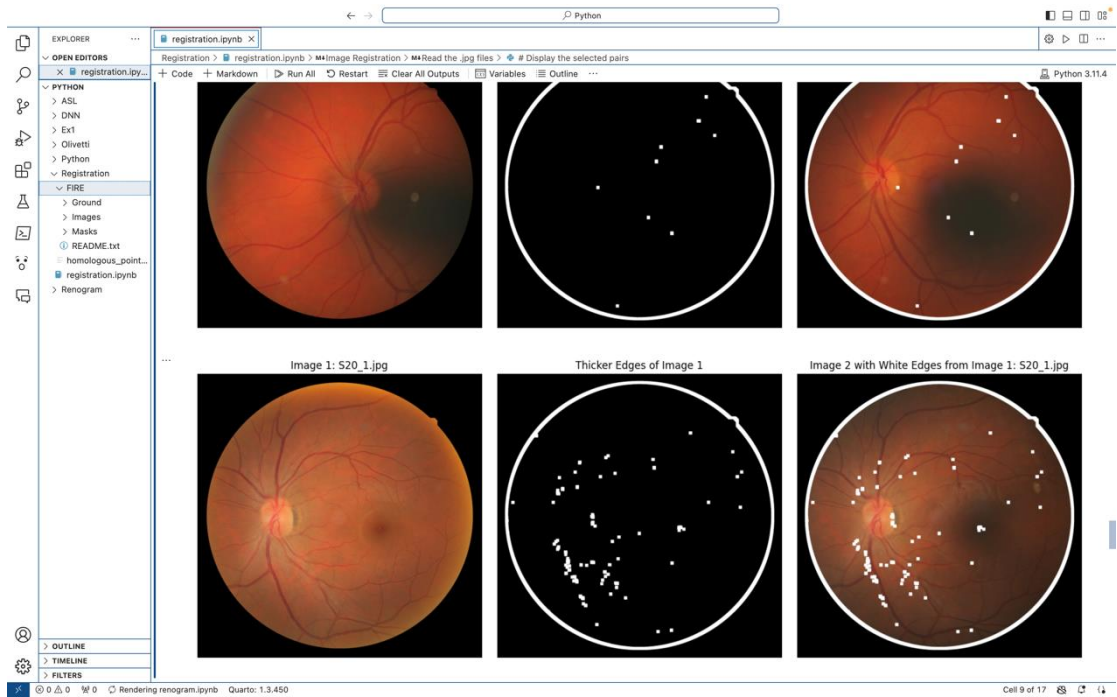
Εικόνα 3.3 Εμφάνιση Ακμών



Εικόνα 3.4 Εμφάνιση Ακμών



Εικόνα 3.5 Εμφάνιση Ακμών



Εικόνα 3.6 Εμφάνιση Ακμών

Διαβάζουμε και δημιουργούμε μία λίστα με τα αρχεία Ground Truth. Το Ground truth στην υπολογιστική όραση αναφέρεται στις πραγματικές τιμές που χρησιμοποιούνται για την εκπαίδευση ή αξιολόγηση των μοντέλων. Στην προκειμένη εργασία τα ground truth είναι αρχεία που περιλαμβάνουν πληροφορίες για τα ομόλογα σημεία μεταξύ των δύο εικόνων. Εκτυπώνονται οι λίστες `src_point` και `dst_point`, όπου

αποτελούνται από τα πραγματικά σημεία της πρώτης εικόνας και τα σημεία στα οποία θέλουμε να αντιστοιχίσουμε τα `src_points` στη δεύτερη εικόνα αντίστοιχα και τις μετατρέπουμε σε `numpy` arrays. Τέλος, χρησιμοποιούμε τη συνάρτηση `cv2.estimateAffine` για να υπολογίσουμε τον αφινικό γεωμετρικό μετασχηματισμό που απαιτείται για να ευθυγραμμίσει το σύνολο των ομόλογων σημείων της πρώτης εικόνας πάνω στην δεύτερη.

```

# 3. Ground Truth for Homologous Points
# 4. Affine Heometric Transformation Matrix

import numpy as np

ground_truth = '/Users/marina/Python/Registration/FIRE/Ground'
ground_truth_filenames = [f'control_points_{filename}.replace("_1.jpg", "_1.2.txt").replace("_2.jpg", "_1.2.txt")' for filename in selected_pairs]

# Save homologous points
output_filename = 'homologous_points.txt'

# Check if the ground truth files exists and display the homologous points
with open(output_filename, 'w') as output_file:
    for image_filename, ground_truth_filename in zip(selected_pairs, ground_truth_filenames):

        image1_path = os.path.join(image_path, image_filename)
        image2_path = os.path.join(image_path, image_filename.replace("_1", "_2"))
        img1 = cv2.imread(image1_path)
        img2 = cv2.imread(image2_path)

        # Read the homologous from the ground truth file
        ground_truth_filepath = os.path.join(ground_truth, ground_truth_filename)

        # Homologous points lists
        src_point = []
        dst_point = []

        if os.path.exists(ground_truth_filepath):
            homologous_points = np.loadtxt(ground_truth_filepath)

            # Homologous points for selected pairs
            for i in range(len(homologous_points)):
                src_point.append(homologous_points[i][0], homologous_points[i][1])
                dst_point.append(homologous_points[i][2], homologous_points[i][3])

            # Display the pairs
            output_file.write(f"Image: {image_filename}, Ground Truth: {ground_truth_filename}")
            output_file.write(f"Homologous Points:\n{homologous_points}")
            output_file.write("\n" * 30)
        else:
            output_file.write(f"Ground Truth file not found: {ground_truth_filename}")

# Print lists
print(f"src_point: {src_point}")
print(f"dst_point: {dst_point}")

```

Εικόνα 4 Ground Truth

```

ground_truth_filepath = os.path.join(ground_truth, ground_truth_filename)

# Homologous points lists
src_point = []
dst_point = []

if os.path.exists(ground_truth_filepath):
    homologous_points = np.loadtxt(ground_truth_filepath)

    # Homologous points for selected pairs
    for i in range(len(homologous_points)):
        src_point.append(homologous_points[i][0], homologous_points[i][1])
        dst_point.append(homologous_points[i][2], homologous_points[i][3])

    # Display the pairs
    output_file.write(f"Image: {image_filename}, Ground Truth: {ground_truth_filename}")
    output_file.write(f"Homologous Points:\n{homologous_points}")
    output_file.write("\n" * 30)
else:
    output_file.write(f"Ground Truth file not found: {ground_truth_filename}")

# Print lists
print(f"src_point: {src_point}")
print(f"dst_point: {dst_point}")
print("\n" * 30)

# Convert lists into numpy array
src_point = np.array(src_point, dtype=np.float32)
dst_point = np.array(dst_point, dtype=np.float32)

# Affine Geometric Transformation
affine_transform, _ = cv2.estimateAffine2D(src_point, dst_point)

if affine_transform is not None:
    print(f"The Affine Transformation:")
    print(affine_transform)
else:
    print(f"Failed to evaluate the affine transformation.")

```

[5] ✓ 1.5s

```

src_point: [[637.0, 626.0], [1679.0, 323.0], [2261.0, 1066.0], [728.0, 1553.0], [1262.0, 2726.0], [1401.0, 1000.0], [2342.0, 2287.0], [1546.0, 863.0], [752.0, 2016.0], [845.0, 1166.0]]
dst_point: [[648.0, 637.0], [1675.0, 327.0], [2259.0, 1056.0], [748.0, 1553.0], [1287.0, 2707.0], [1418.0, 1790.0], [2349.0, 2262.0], [1551.0, 862.0], [778.0, 2010.0], [860.0, 1170.0]]

The Affine Transformation:
[[ 9.88019642e-01  1.00169925e-02  1.36666626e+01]
 [-1.02188822e-02  9.88611510e-01  2.49764973e+01]]

```

Εικόνα 4.1 Affine Geometric Transformation

Στο σημείο αυτό επαναλαμβάνουμε τα παραπάνω βήματα της μετατροπής των λιστών σε πίνακες και το διάβασμα των ομολόγων σημείων για το κάθε ζεύγος. Έπειτα, χρησιμοποιείται η συνάρτηση `estimateAffinePartial2D` για την εκτίμηση του αφινικού γεωμετρικού μετασχηματισμού μεταξύ των δύο ομολόγων σημείων και τελικά οπτικοποιούμε τον μετασχηματισμό των δύο εικόνων, δηλαδή την διαφορά τους.

```

# 5. Apply the transformation to one image and one technique visualization to see the accuracy of the transformation.

# Convert point lists to numpy array
src_point = np.array(src_point)
dst_point = np.array(dst_point)

# Calculate Affine geometric transformation
affine_transform = cv2.estimateAffinePartial2D(src_point, dst_point)[0]

if affine_transform is not None and affine_transform.shape == (2, 3):
    print("Affine Transformation:")
    print(affine_transform)
else:
    print("Δεν ήταν δυνατή η εκτίμηση του αφινικού μετασχηματισμού.")

# Εφαρμογή του μετασχηματισμού στην πρώτη εικόνα
transformed_image = cv2.warpAffine(img1, affine_transform, (img2.shape[1], img2.shape[0]))

# Οπτικοποίηση του μετασχηματισμού
difference_image = cv2.absdiff(img2, transformed_image)

# Εμφάνιση των εικόνων
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(img1, cv2.COLOR_BGR2RGB))
plt.title('Original Image')

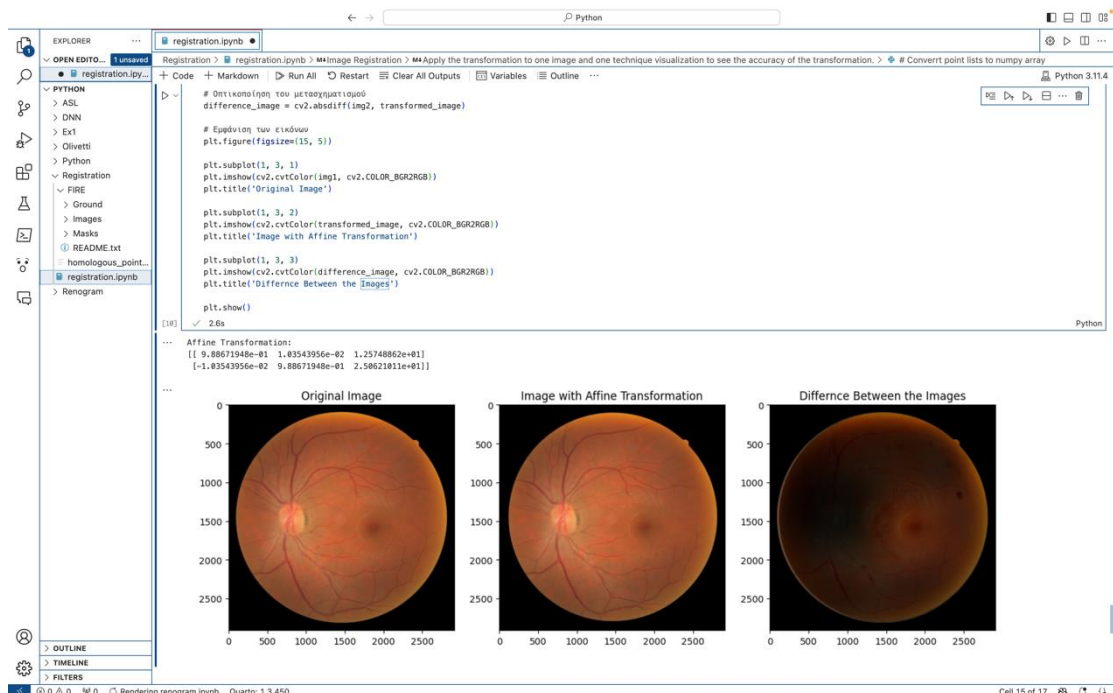
plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(transformed_image, cv2.COLOR_BGR2RGB))
plt.title('Image with Affine Transformation')

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(difference_image, cv2.COLOR_BGR2RGB))
plt.title('Difference Between the Images')

plt.show()

```

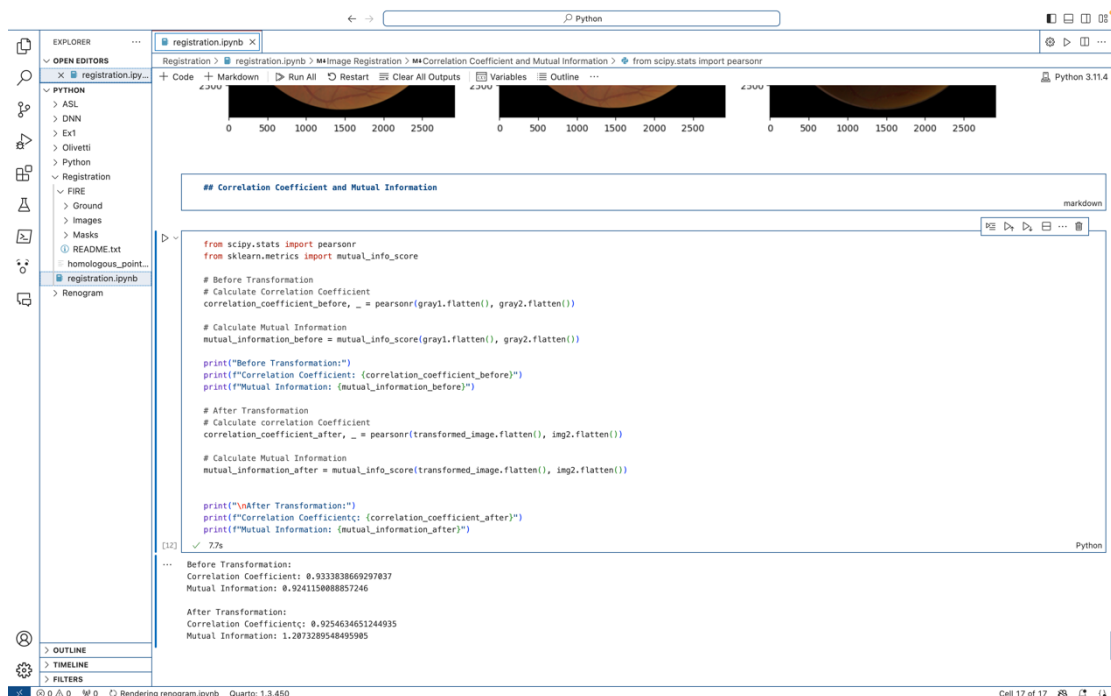
Εικόνα 5.1 Affine Transformation



Εικόνα 5.2 Difference Between Images

Τέλος, χρειάζεται να υπολογίσουμε τον συντελεστή συσχέτισης και αμοιβαίας πληροφορίας πριν και μετά την εφαρμογή του αφινικού γεωμετρικού μετασχηματισμού. Κρίνεται απαραίτητη η χρήση της συνάρτησης Pearson και της `mutual_info_score` για τον υπολογισμό του συντελεστή συσχέτισης και της αμοιβαίας πληροφορίας αντίστοιχα.

Παρατηρούμε πως πριν τον αφινικό γεωμετρικό προσανατολισμό έχουμε υψηλό συντελεστή συσχέτισης και υψηλή αμοιβαία πληροφορία κάτι που μας υποδεικνύει πως υπάρχει υψηλή γραμμική σχέση μεταξύ των εικόνων και οι εικόνες μας περιέχουν κοινή πληροφορία. Μετά τον μετασχηματισμό, ο συντελεστής συσχέτισης παραμένει υψηλός, ωστόσο έχει υποστεί μικρή μείωση αλλά εξακολουθεί να υποδεικνύει ότι υπάρχει κάποια συσχέτιση μεταξύ των εικόνων. Το αποτέλεσμα αυτό είναι αναμενόμενο, καθώς ο αφινικός γεωμετρικός μετασχηματισμός δεν είναι απαραίτητα γραμμικός. Τέλος, μετά τον αφινικό γεωμετρικό μετασχηματισμό η αμοιβαία πληροφορία είναι αυξημένη κάτι που μπορεί να υποδηλώνει ότι έχει αυξηθεί η κοινή πληροφορία μεταξύ των εικόνων.



Εικόνα 6 Correlation Coefficient and Mutual Information

2.1 Συμπλήρωση Πίνακα

Ζεύγος Εικόνων	Πίνακας Μετασχηματισμού			Mutual Information Before	Mutual Information After	Correlation Coefficient Before	Correlation Coefficient After
S20_1 S20_2	9.88671948e-01	1.03543956e-02	1.25748862e+01	0.9241	1.2073	0.9334	0.9255
	-1.03543956e-02	9.88671948e-01	2.50621011e+01				

3. Βιβλιογραφία

- [1]. Κ. Δελήμπασης, Διαλέξεις μαθήματος e-class, Πανεπιστήμιο Θεσσαλίας, Λαμία, 2023-2024
- [2]. <https://projects.ics.forth.gr/cvrl/fire/>
- [3]. <https://www.maio-journal.com/index.php/MAIO/article/view/42/90>
- [4]. <https://www.mathworks.com/discovery/affine-transformation.html>
- [5]. <https://www.geeksforgeeks.org/python-opencv-canny-function/>